Arduino: microcontrollore, pin, GND e il primo programma

Lezione 5 — 1MAK, SST-INF-01c — La scheda Arduino Uno, i pin digitali, il riferimento GND, USB e alimentazione, Tinkercad, pinMode, digitalWrite, delay, setup e loop

13/10/2026

1 Da dove ripartiamo

Nota

Torniamo al discorso iniziato le prime volte parlando di **Arduino**: la **scheda elettronica** che ci servirà per imparare a programmare. Il docente ha iniziato a **produrre** una scheda con i componenti che useremo nelle prossime lezioni. Prima di passare alla pratica servono alcune cosettine senza le quali non si riesce proprio a lavorare: oggi vediamo come è fatta la scheda e scriviamo, in simulazione, il **primo programma**.

2 Il cervello della scheda: il microcontrollore

Definizione

Il **microcontrollore** è il **cervello** della scheda elettronica: il componente che, in base al **programma** che scriviamo noi, comanda tutto il resto.

- Nel mondo esistono **migliaia di modelli** diversi di microcontrollori (decine, centinaia di migliaia).
- Quello montato sull'**Arduino Uno**, il modello base e "storico", si chiama **ATmega328P**. Poi sono uscite tante altre versioni di Arduino, con processori più o meno potenti.
- È un **chip nero** con tante **"zampette"** metalliche, i **pin** (o piedini, terminali).

Definizione

Che cosa fa un pin. Ogni terminale metallico del microcontrollore è un **filo** che possiamo **pilotare dal programma**: possiamo metterci una **tensione** oppure no.

- Arduino lavora a **5 volt**. Rispetto a un piedino di riferimento che sta a **0 V**, un pin può trovarsi a **+5 V** oppure a **0 V**.
- Quindi un pin può essere **acceso o spento**: quando è **0** sta a 0 V, quando è **1** il microcontrollore ci mette **5 V**.
- La sequenza di accensioni e spegnimenti dipende dal **programma** che scriviamo.

3 I connettori sul bordo e i pin digitali

I piedini del microcontrollore sono **riportati sul bordo** della scheda, su dei connettori con dei buchini in cui si possono infilare dei fili. Lì colleghiamo resistori, lampadine, pulsanti, servomotori… tutto quello che vi passa per la testa: è la nostra interfaccia verso l'esterno. Quale buchino sia collegato a quale zampetta del chip, per ora, ci interessa poco.

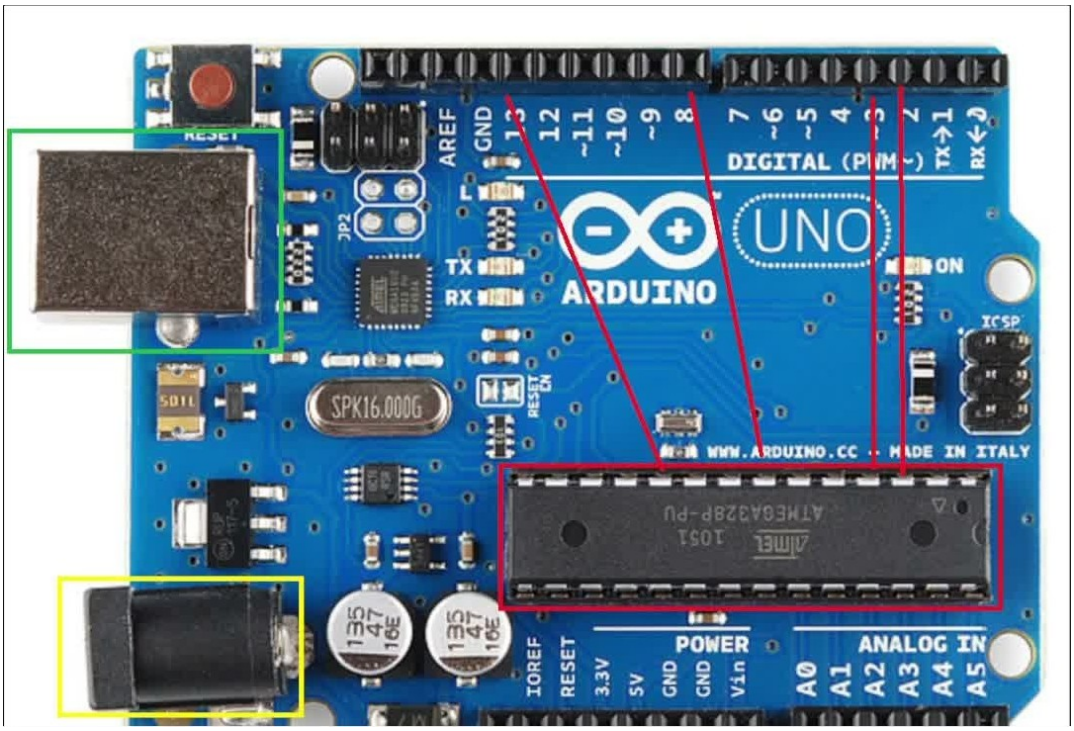


Figura 1: La scheda Arduino Uno annotata in classe. In rosso il microcontrollore ATmega328P e alcuni dei collegamenti tra i suoi piedini e i connettori in alto; in verde la porta USB; in giallo il connettore di alimentazione.

Definizione

I pin digitali da 0 a 13. In alto, partendo da destra, i pin sono **numerati da 0 a 13**. Quanti sono in tutto? Non 13: sono **14**, perché si parte da **zero**.

Nota

In informatica lo **zero** si scrive spesso con una **barra in mezzo** (Ø), per non confonderlo con la lettera O. Sulla stampa della scheda lo zero è un po' rovinato, ma è lo zero.

Attenzione

I pin 0 e 1 (RX e TX). Accanto ai pin 0 e 1 c'è scritto **RX** e **TX**: servono alla scheda per **comunicare con il computer** e per scaricarci dentro il programma, che viaggia proprio attraverso quei fili. Si possono usare anche per altro, ma bisogna stare attenti a cose che ancora non siamo in grado di apprezzare: se non le facciamo, qualcosa non funziona e non si capisce perché. **Regola per ora:** i pin 0 e 1 **non li usiamo**, li lasciamo sempre liberi. Con il tempo capiremo come sfruttarli.

4 GND: il riferimento di tensione

Definizione

Il piedino **GND** (dall'inglese *ground*) è il nostro **riferimento di tensione**: per la scheda è il punto a **0 volt**.

- Sulla scheda ce ne sono **tre**: uno in alto, accanto al pin 13, e due in basso.
- Se pensiamo a un generatore, il GND è il suo **negativo**: il terminale a potenziale **0 V**.
- Quando diciamo che un punto "vale 5 V", intendiamo **5 V rispetto a GND**.

Ricordate che **elettronica e firmware vanno insieme**: anche parlando di programmazione ci entrano dentro cose di elettronica. Negli **schemi elettrici** il filo di GND ha un suo **simbolo**. In un circuito con un generatore e un resistore, i simboli di massa dicono che i punti dove sono disegnati sono **collegati tra loro** e valgono **0 V**, anche senza disegnare il filo che li unisce. Se il filo in alto sta a 5 V, tutto quello che è collegato lì sta a 5 V **rispetto a GND**.

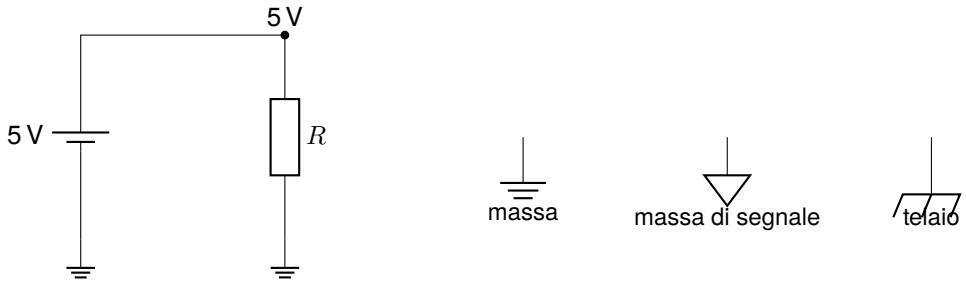


Figura 2: A sinistra: un generatore da 5 V e un resistore; i due simboli di massa dicono che i due punti in basso sono collegati insieme e stanno a 0 V. A destra: tre simboli che si trovano negli schemi per indicare il riferimento a 0 V (ricostruzione: i simboli disegnati alla lavagna non sono visibili nella registrazione).

Attenzione

Non c'è un simbolo unico. Negli schemi elettrici trovate **simboli diversi** per il GND, dipende da chi disegna. Alcuni simboli simili servono per dire **altre cose**: per esempio il **simbolo di terra** (il collegamento all'impianto di terra) non è la stessa cosa del riferimento a 0 V del circuito. Bisogna **imparare a riconoscerli**; a noi per ora bastano tre.

Attenzione

Regola pratica. Ogni volta che colleghiamo un circuito ad Arduino, il **GND** deve arrivare a **tutte le periferiche** che usiamo: è il riferimento di tensione comune.

5 USB e alimentazione

- La **porta USB** (in verde nella figura) serve per **collegare la scheda al computer**: la scheda comunica con il computer e il computer ci **scarica dentro il programma**.

- Il **connettore di alimentazione** (in giallo) serve ad **alimentare** la scheda senza computer.

Definizione

Il computer serve solo per programmare. Una volta che il programma è stato scaricato nella **memoria del microcontrollore**, il computer **non serve più**. Esempio: voglio costruire un **drone** (o un sistema per gestire l'irrigazione di un vigneto). Il drone non può volare collegato con un cavo al computer. Quindi:

1. collego la scheda al computer con il **cavo USB** e scarico il programma;
2. verifico che funzioni;
3. stacco il cavo USB;
4. alimento la scheda dal **connettore di alimentazione**, per esempio con una **batteria**: la scheda si accende e, se l'abbiamo programmata correttamente, funziona.

Quando invece sperimentiamo al computer (per esempio per far lampeggiare una lampadina), il cavo USB porta già anche l'**alimentazione**: la scheda la prende da lì.

6 Due modi di lavorare: scheda vera e simulatore

- Nei prossimi giorni lavoreremo con la **scheda vera**: Arduino, un po' di **resistori**, di **lampadine** e di **pulsanti**, con cui faremo dei progettini per imparare a usarlo.
- Prima, per non **bruciare una scheda** collegandola male, si lavora in **simulazione**, con un simulatore molto semplice che abbiamo già usato: **Tinkercad**.

Entrare in Tinkercad

1. Accendete il computer e fate l'accesso a **Chrome** con il vostro **account** (in alto a destra).
2. Andate sul sito **tinkercad.com** e fate **Accedi**.
3. Chi si era già iscritto l'altra volta sceglie l'accesso con **account personale** e poi entra con l'**account Google**; chi non l'ha mai fatto si deve prima **iscrivere**.
4. La prossima volta sarà più veloce. Il docente vede i vostri monitor: se qualcuno ha problemi, chiama e si risolve insieme.

Il primo circuito

1. In alto a destra: **Crea** → **Circuito**.
2. Dall'elenco dei componenti si prende **Arduino Uno R3**: si tiene premuto e lo si **trascina** nel piano di lavoro. Nel simulatore teniamo **solo l'Arduino**.
3. In alto si preme **Codice**. La prima cosa da fare, **ogni volta**, è cambiare da **Blocchi** a **Testo** e confermare con **Continua**.
4. Nel codice compare tre volte la parola `LED_BUILTIN`: in tutti e **tre i posti** la si cancella e si scrive al suo posto il numero **13**.

5. Si preme **Avvia simulazione**: la lampadina **L** sulla scheda **lampeggia**.

7 Il primo programma

Esempio

Il codice in Tinkercad (è un codice in C; Tinkercad lo indica come C++):

```
// C++ code
//
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(13, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Il **13** è il numero del **pin** che vogliamo comandare, cioè il terminale 13. Perché proprio quello? Perché sulla scheda Arduino il pin 13 è collegato anche alla piccola lampadina **L**. Al programma stiamo dicendo: **accendi** la lampadina, poi **spegnila**, e così via.

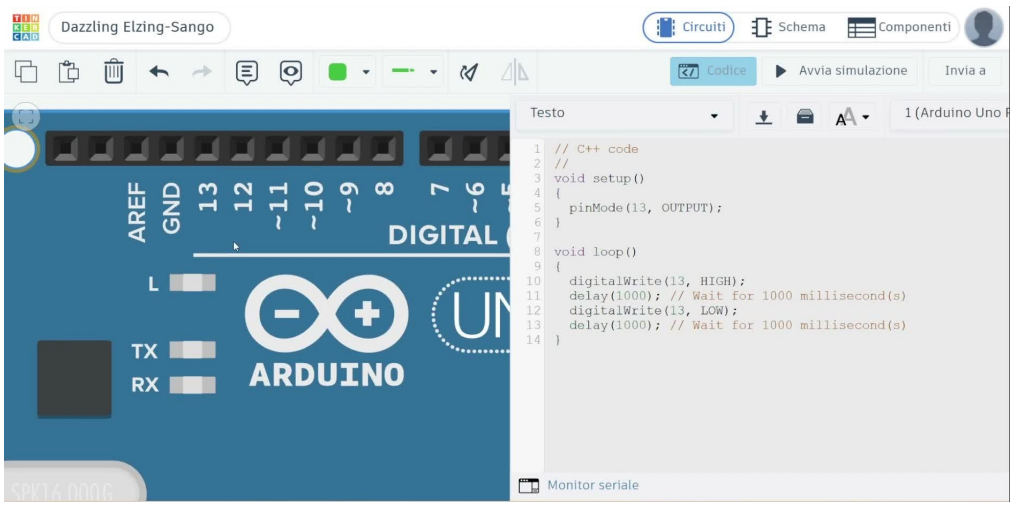


Figura 3: Tinkercad: a sinistra l'Arduino Uno con la lampadina L (vicino al pin 13), a destra il codice in modalità Testo, con il numero 13 al posto di `LED_BUILTIN`. In alto i pulsanti Codice e Avvia simulazione.

7.1 HIGH e LOW

Definizione

- `HIGH` vuol dire **alto**: per noi **acceso**, a livello di bit **1** (il pin va a 5 V).
- `LOW` vuol dire **basso**: per noi **spento**, a livello di bit **0** (il pin va a 0 V).

HIGH e LOW sono i due **stati** possibili di un filo, cioè di un pin: acceso o spento.

7.2 digitalWrite: scrivere lo stato di un pin

Definizione

Il comando `digitalWrite` serve a **scrivere** lo stato di un pin: decide se il pin è **alto o basso**, acceso o spento.

```
digitalWrite(13, HIGH);   // pin 13 acceso
digitalWrite(13, LOW);    // pin 13 spento
```

Tra le tonde: prima il **numero del pin**, poi una virgola, poi `HIGH` o `LOW`. La riga termina con il **punto e virgola**.

Esempio

Domande dalla classe.

- *Se scrivo 6 al posto di 13?* Il programma funziona lo stesso: il pin 6 si accende e si spegne. Ma nel simulatore **non si vede niente**, perché la lampadina è collegata solo al **13**.
- *Il segnetto accanto ad alcuni numeri (11, 10, 9...)?* È la **tilde** (~): indica una funzione più evoluta che vedremo più avanti. Per ora interessano solo i **numeri**.

Nota

Questi **comandi** vanno **imparati**: li trovate segnati nel manuale e nelle dispense, ma non si può andare a cercarli ogni volta.

7.3 pinMode: ingresso o uscita?

Arduino si può disegnare come una **scatola**. Questa scatola può **decidere** qualcosa e comandare delle azioni all'esterno, oppure **ascoltare** ed essere influenzata da qualcuno all'esterno.

Esempio

Bocca e orecchie. Per dire qualcosa a un'altra persona usate la **bocca**, la voce: il suono esce dal vostro corpo. Per capire quello che dice l'altro usate le **orecchie**: l'informazione entra. Non si parla con le orecchie e non si ascolta con la bocca: sono due "periferiche" diverse.

- Un pin configurato come **orecchio** è un **ingresso**: in inglese `INPUT`.
- Un pin configurato come **bocca**, con cui la scheda dice al mondo esterno cosa fare, è un'**uscita**: `OUTPUT`.

Definizione

Prima di usare un pin bisogna dire se è un **ingresso o un'uscita**: non posso dire di accendere o spegnere il pin 13 se è un orecchio. Si usa il comando `pinMode`: tra le tonde il **numero del pin** e poi `INPUT` oppure `OUTPUT`.

```
pinMode(13, OUTPUT);   // il pin 13 e' un'uscita
```

Nel codice di Tinkercad questo comando c'è già, alla **riga 5**.

Attenzione

Quando accendete la scheda, la prima cosa che succede è che **tutti i pin** sono **ingressi**: tutte orecchie. Se vi **dimenticate il `pinMode`**, la lampadina **non funziona** come deve.

Nota

Due comandi da ricordare a memoria.

- `pinMode`: dice la **direzione** dell'informazione, cioè se è Arduino a decidere cosa fare (uscita) o se Arduino deve sentire cosa hanno da dire gli altri (ingresso).
- `digitalWrite`: per un pin di **uscita**, decide se è **alto o basso**.

7.4 I due blocchi: setup e loop

Definizione

Ogni programma Arduino deve contenere **due blocchi di codice**:

```
void setup()
{
  // qui il codice del setup
}

void loop()
{
  // qui il codice del loop
}
```

- Ogni blocco è un **contenitore di codice**: i suoi limiti sono le graffe, e dentro si scrive tutto quello che serve.
- Nel nostro programma il `setup` contiene solo il `pinMode(13, OUTPUT);`, il `loop` contiene **4 righe** di codice.

- I due blocchi **devono esserci sempre**: non si può scrivere un programma Arduino in cui ne manca uno.
- Ce ne deve essere **uno solo** per tipo: non si possono scrivere due `setup` o due `loop`.

7.5 delay: aspettare

Definizione

Tra l'accensione e lo spegnimento c'è il comando **`delay`**: dice alla macchina di **fermarsi** per il numero di **millisecondi** scritto tra le tonde.

```
delay(1000);   // aspetta 1000 millisecondi
```

Quanto sono **1000 millesimi di secondo**? Un **secondo**: $1000 \times \frac{1}{1000}\,\text{s} = 1\,\text{s}$.

Esempio

Che cosa fa il programma, passo per passo. Accende la lampadina, **aspetta un secondo**, la spegne, **aspetta un secondo**, poi ricomincia.

Attenzione

Setup una volta, loop per sempre.

- Il **`setup`** viene eseguito **una sola volta**, all'inizio, quando si accende la scheda.
- Finito il setup, il **`loop`** viene **ripetuto continuamente**: finché non si stacca l'alimentazione, **non si ferma mai**.

8 Cosa ricordare di questa lezione

- Il **microcontrollore** è il cervello della scheda; su Arduino Uno è l'**ATmega328P**.
- Arduino lavora a **5 V**: un pin acceso (1) sta a 5 V, spento (0) a 0 V.
- I pin digitali vanno da **0 a 13**, quindi sono **14**; i pin **0 e 1** (RX, TX) servono a comunicare con il computer: per ora **non si usano**.
- **GND** è il riferimento a **0 V**; deve arrivare a **tutte le periferiche**. Negli schemi ha simboli diversi.
- La **USB** serve a programmare (e alimenta durante le prove); dopo, la scheda può funzionare da sola con una **batteria**.
- In **Tinkercad**: Codice → da Blocchi a **Testo**; `LED_BUILTIN` → **13**.
- **`pinMode(pin, OUTPUT/INPUT)`**: uscita (bocca) o ingresso (orecchio).
- **`digitalWrite(pin, HIGH/LOW)`**: alto = acceso = 1, basso = spento = 0.
- **`delay(1000)`**: aspetta **1 secondo**; **`setup`** una volta sola, **`loop`** per sempre (uno per tipo, entrambi obbligatori).